DSCI 551: Spring 2026 Final Project

Learning Analytics Backend with DuckDB: Columnar Storage Architecture for Educational Aggregation Workloads

Author: Rosie Xiu, lxiu@usc.edu
Course: DSCI 551
University of Southern California
May 8, 2026

Table of Contents

1 Introduction and Motivation
2 Database System Overview
3 Internal Architecture
3.1 Storage Engine and Compression
3.2 Indexing
3.3 Query Execution Pipeline
3.4 Distribution Model
4 Application Design
4.1 Dataset Schema
4.2 Query Categories
4.3 Technology Stack
5 Mapping Internals to Application Behavior
5.1 Column Pruning in Action (All Queries)
5.2 Vectorized Scan versus Tuple-at-a-Time (Q1–Q3)
5.3 Perfect Hash Aggregation (Q1)
5.4 Top-K Optimization (Q3)
5.5 Hash Join Pipeline (Q4–Q5)
5.6 Benchmark Methodology
5.7 Benchmark Results
6 Comparison with MySQL and MongoDB
6.1 Empirical Comparison with MySQL 9.6
6.2 Theoretical Comparison with MongoDB
7 Limitations, Lessons Learned, and Future Work
7.1 Limitations
7.2 Lessons Learned
7.3 Future Work
8 Project Links
9 References

1 Introduction and Motivation

Modern learning analytics platforms collect fine-grained trace data from virtual learning environments (VLEs), capturing every student click, page view, quiz attempt, and forum interaction. This clickstream data is inherently multidimensional: each event has a student identifier, a timestamp, a resource type, and an associated course module. Over an academic term, a single institution can accumulate tens of millions of such records. The analytical queries run against this data are overwhelmingly aggregation-heavy: group-by counts over time windows, average interaction rates per student across modules, and correlation patterns between engagement and assessment outcomes.

Traditional relational databases such as MySQL rely on row-oriented storage (InnoDB). Row stores excel at transactional workloads that frequently insert, update, retrieve individual rows by primary key. But they become inefficient for analytical workloads that read millions of rows, scan a few columns, aggregate over groups. When a row store processes a query like "sum the clicks per student per day," it must pull every column of every row into memory, even though only three columns are needed. This I/O overhead translates directly into wall-clock time for the analyst.

DuckDB is a column-oriented OLAP database designed specifically for analytical workloads. It stores each column as a contiguous block on disk, so a query that touches three columns reads only those three blocks. DuckDB also employs vectorized query execution, where data is processed in small batches (vectors of 2048 values) that fit in CPU registers, maximizing cache utilization and enabling SIMD parallelism. For educational data science, where the workload is exploratory and aggregation-driven, DuckDB offers a compelling alternative to both traditional RDBMS and big-data frameworks [1] [2].

This project builds a learning analytics backend using DuckDB over the Open University Learning Analytics Dataset (OULAD). The dataset contains 10.6 million VLE interaction records linked to student demographic data, assessment scores, and course registration information across 7 relational tables. I execute five analytical queries that mirror real-world educational data analysis tasks: engagement trends over a course term, comparison of interaction patterns across modules, high-engagement identification, multi-table correlation between interactions and assessment outcomes, and aggregate interaction summaries by resource type. I analyze DuckDB's internal execution plans to understand why columnar processing delivers such dramatic performance advantages. I then benchmark DuckDB against MySQL 9.6 on identical queries and hardware, documenting speedups of 192x to 618x. Finally, I discuss limitations, lessons learned, and the implications of columnar storage for learning analytics infrastructure at scale [3].

2 Database System Overview

DuckDB is an embedded, in-process OLAP database management system written in C++. It is designed to run within the same process space as its host application (e.g., a Python script or Jupyter notebook), eliminating the network overhead associated with client-server architectures. DuckDB targets single-node analytical performance that approaches or exceeds specialized OLAP engines while requiring zero configuration and no separate server process [1, 2].

System specifications:

- Database engine: DuckDB 1.2 (Python duckdb package 1.2.1)
- Deployment model: Embedded, in-process via Python DB-API 2.0
- Storage model: Column-oriented (PAX-style on-disk format)
- Implementation language: C++ core with Python bindings
- License: MIT (open source)
- Platform: macOS 15.2

DuckDB occupies a distinct position in the database ecosystem. It is not a competitor to MySQL or PostgreSQL for transactional workloads. Instead, it competes with systems like ClickHouse, MonetDB, and Apache Druid for analytical query processing. Its key differentiator is its embedded nature: no server setup, no network configuration, no separate process management. This makes it uniquely suited for data science and research workflows where analysts want database performance without infrastructure overhead.

3 Internal Architecture

3.1 Storage Engine and Compression

DuckDB stores data in a custom columnar on-disk format. Each column of a table is stored as a separate set of compressed data blocks. When a query references only a subset of columns, DuckDB reads only those column blocks from disk. For a table with 7 columns, a query referencing 2 columns reads approximately 29% of the data. This column pruning is the single largest factor in DuckDB's performance advantage over row-oriented systems.

We loaded the 454 MB studentVle CSV (10.6M rows, 7 columns) into DuckDB and measured the on-disk size. DuckDB stored the same data in approximately 62 MB, a 7.3x compression ratio. This compression comes from multiple techniques applied automatically per column based on data characteristics: [5].

- Dictionary encoding: Applied to low-cardinality string columns. The code_module column, for example, has only 7 distinct values across 10.6M rows. DuckDB stores each unique value once and references it through a small integer, reducing storage from CSV's raw text to a compact token map.

- Run-length encoding (RLE): Applied to sorted or repetitive integer sequences. The date column, when stored in order, contains long runs of the same value (many students clicking on the same day). RLE stores these as (value, count) pairs instead of individual integers.
- Lightweight integer compression: Applied to high-cardinality numeric columns. DuckDB uses frame-of-reference (FOR) encoding with bit-packing, storing offsets from a base value rather than full 64-bit integers. The sum_click column, which has values between 0 and 498, fits in 9 bits per entry instead of 64.
- Adaptive codec selection: DuckDB samples column data during ingestion and selects the optimal compression scheme per column segment. Different segments of the same column may use different codecs based on local data distribution.

For comparison, MySQL's InnoDB row-based storage required approximately 612 MB for the same data with default settings, and 932 MB with secondary indexes on the columns used in GROUP BY clauses. DuckDB achieves a 10x to 15x storage reduction over MySQL without any index maintenance overhead.

3.2 Indexing

DuckDB does not use traditional B+ tree indexes. Instead, it relies on zonemaps (min-max statistics stored per row group of approximately 500,000 rows). When a query includes a range predicate, DuckDB checks the zonemap for each row group and skips groups whose min-max range does not intersect the query predicate. For the aggregation-heavy queries in this project, which perform full table scans grouped by key columns, zonemaps provide limited benefit because no row groups are skipped. The performance advantage comes from column pruning, vectorized execution, and the absence of index maintenance overhead during bulk data loading.

This index-light approach is appropriate for analytical workloads where most queries scan large fractions of the data. Maintaining B+ tree indexes for all GROUP BY columns would consume additional storage (as seen in the MySQL comparison) and add write overhead without improving scan-based aggregation performance.

3.3 Query Execution Pipeline

Duck DB's query execution is fundamentally vectorized. Each physical operator processes data in vector batches of 2048 values rather than one tuple at a time. The execution pipeline consists of operators connected through a pull-based model (each operator requests the next vector from its child), but within each operator, processing is loop-based over vectors.

Key execution operators observed in our EXPLAIN plans:

- SEQ_SCAN (sequential scan): Reads projected columns from disk in vector batches. Due to columnar storage, only the columns referenced in the query are deserialized. EXPLAIN plans show that Q1 scans only 2 columns (date, sum_click) despite the table having 7 columns.

- PERFECT_HASH_GROUP_BY: A specialized hash aggregation variant used when the number of distinct group keys fits in CPU cache. Q1's GROUP BY date produces 299 groups, which DuckDB detects as cacheable and uses a contiguous hash table instead of a general-purpose hash map. This avoids pointer chasing and hash collisions.
- HASH_GROUP_BY: The general-purpose hash aggregation used when group key cardinality exceeds cache capacity. Used in Q2 and Q3, which group by code_module (7 groups) and id_site (6,268 groups) respectively. The hash table is built using vectorized key hashing, processing 2048 tuples per call.
- TOP_N: A bounded heap operator used in Q3 (GROUP BY + ORDER BY + LIMIT 20). Instead of sorting all 6,268 groups, DuckDB maintains a min-heap of 20 elements, inserting each new group and popping the smallest if the heap exceeds 20. This reduces sort complexity from O(N log N) to O(N log K) where K = 20.
- HASH_JOIN: Used in Q4 (4-table JOIN) and Q5 (5-table JOIN). DuckDB builds a hash table on the smaller input (build side) and probes with the larger input (probe side) in vectorized batches. The optimizer automatically selects which table to use as the build side based on cardinality estimates.
- Internal compression operators: EXPLAIN plans show compress/decompress operators (e.g., __internal_compress_integral_usmallint, __internal_decompress_integral_bigint) inserted into the plan. These operators handle on-the-fly conversion between compressed on-disk format and vectorized in-memory representation.

3.4 Distribution Model

DuckDB is designed as a single-node analytical engine. It does not support distributed query execution across multiple machines, shared-nothing parallelism, or cluster deployment. Each process opens the database file exclusively, preventing concurrent multi-user access. This is by design: DuckDB targets embedded analytical workloads where a single analyst runs complex queries against local data. For the learning analytics use case 10.6M rows fit comfortably in memory on modern hardware and the simplicity of in-process execution outweighs the benefits of distributed parallelism.

4 Application Design

4.1 Dataset Schema

The application is implemented as a Jupyter notebook that uses DuckDB to process and analyze the OULAD dataset, a real-world educational dataset from the Open University. The OULAD relational schema consists of seven interconnected tables:

- courses: 7 courses with module and presentation metadata.
- vle: 6,364 VLE activity definitions (resource types, URLs, activity categories).
- studentInfo: 32,593 student demographic records (gender, region, education level, age band).

- studentRegistration: 32,593 registration and unregistration records with timestamps.
- assessments: 206 assessment definitions with weight and cutoff dates.
- studentAssessment: 173,912 student-assessment score records.
- studentVle: 10,655,280 VLE clickstream events, the core analytical fact table.

4.2 Query Categories

The notebook implements five analytical query categories:

Q1 — Daily Engagement Aggregation (GROUP BY date): SELECT date, SUM(sum_click) AS total_clicks FROM student_vle GROUP BY date ORDER BY date. This single-table aggregation over 10.6M rows represents the most common analytics pattern: scanning one fact table, grouping by a temporal dimension, and computing a sum.

Q2 — Module-Level Activity (GROUP BY module, two aggregates): SELECT code_module, AVG(sum_click) AS avg_clicks, COUNT(*) AS click_records FROM student_vle GROUP BY code_module. Tests the engine's ability to compute multiple aggregates over the same grouping key in a single scan.

Q3 — Top-K Resource Analysis (GROUP BY + ORDER BY + LIMIT): SELECT id_site, SUM(sum_click) AS total_clicks FROM student_vle GROUP BY id_site ORDER BY total_clicks DESC LIMIT 20. Tests top-k optimization, where the database must avoid sorting the full result set.

Q4 — Demographics × Performance (4-table JOIN): Joins student_vle, studentInfo, studentAssessment, and assessments across student, module, and presentation keys. Reveals relationships between engagement, demographics, and final scores.

Q5 — Dropout and Engagement (5-table JOIN): Joins student_vle, studentInfo, studentRegistration, and courses with code_module and code_presentation keys. Correlates VLE activity patterns with outcome predictions.

4.3 Technology Stack

- Database engine: DuckDB (v1.x, via Python duckdb package)
- Query interface: DuckDB Python DB-API 2.0
- Notebook environment: Jupyter Notebook and VS Code
- Comparison database: MySQL 9.6 (installed via Homebrew, CLI)
- Visualization: Matplotlib for engagement trend plots
- Hardware: macOS 15.2, Apple M4 Max, 36 GB RAM, SSD storage

5 Mapping Internals to Application Behavior

This section connects each application query to the DuckDB internal mechanisms that determine its performance.

5.1 Column Pruning in Action (All Queries)

Each query selects only 2 to 3 columns from the 7-column student_vle table. DuckDB's columnar storage means only these columns are decompressed and materialized into vectors. Q1 reads only date and sum_click; Q2 reads only code_module and sum_click; Q3 reads only id_site and sum_click. MySQL, by contrast, reads all 7 columns from every row because InnoDB's page structure interleaves columns within each row.

5.2 Vectorized Scan versus Tuple-at-a-Time (Q1–Q3)

When DuckDB executes SEQ_SCAN on student_vle, it reads compressed column blocks and decompresses them into vector batches of 2048 values. Each batch passes through the aggregation pipeline as a unit. MySQL's InnoDB uses the traditional iterator model where each operator produces and consumes one row at a time, incurring function call overhead per row. For 10.6M rows, that overhead alone accounts for a significant portion of the performance gap.

5.3 Perfect Hash Aggregation (Q1)

Q1's EXPLAIN plan reveals PERFECT_HASH_GROUP_BY, triggered because date has only 299 distinct values. DuckDB allocates a contiguous array of 299 hash table slots, one per group. This eliminates hash collisions, avoids redirection through pointer chains, and keeps the hash table in CPU L2 cache. The result is a one-pass aggregation that completes in under 4 ms.

5.4 Top-K Optimization (Q3)

Q3's EXPLAIN plan shows TOP_N instead of ORDER BY + LIMIT. TOP_N maintains a min-heap of 20 entries. As each of the 6,268 groups is emitted by the hash aggregation, it is inserted into the heap. The heap size never exceeds 21 entries, so the final sort is O(K log K) with K=20 instead of O(N log N) with N=6,268.

5.5 Hash Join Pipeline (Q4–Q5)

The multi-table JOIN queries demonstrate DuckDB's hash join pipeline. Smaller tables (studentInfo, 32K rows; assessments, 206 rows) are built into hash tables. The larger fact table (studentVle, 10.6M rows) probes these hash tables in vectorized batches. DuckDB's optimizer chooses the build order automatically, ensuring the smallest tables are hashed first.

5.6 Benchmark Methodology

All queries were run 5 times on identical hardware (Apple M4 Max, 36 GB RAM, SSD, macOS 15.2). DuckDB and MySQL 9.6 both used default configurations. The DuckDB database was created fresh from CSV by running setup.py. Before measurements, the database was warmed by executing one query pass. Each query was then executed 5 times with the minimum time reported, following the standard practice of using the fastest run to approximate best-case performance. MySQL used InnoDB with default buffer pool settings (128 MB) and auto-increment primary keys.

5.7 Benchmark Results

Table 1

DuckDB vs MySQL 9.6 Benchmark Results (10.6M rows)

The results confirm the theoretical performance advantages of DuckDB's columnar architecture. Speedups range from 4.3x to 618x depending on query type.

Query	DuckDB	MySQL 9.6	Speedup
Q1 (GROUP BY date)	3.8 ms	2,370 ms	618x
Q2 (GROUP BY module)	12 ms	2,956 ms	247x
Q3 (GROUP BY + LIMIT)	14 ms	2,716 ms	192x
Q4 (4-table JOIN)	259 ms	5,122 ms	20x
Q5 (5-table JOIN)	694 ms	3,010 ms	4.3x

The speedup is largest for single-table aggregation queries (Q1–Q3), where column pruning gives DuckDB its greatest advantage. For Q1, DuckDB reads 2 columns while MySQL reads all 7, a 3.5x I/O reduction, but the actual speedup is 618x. The additional difference comes from vectorized execution, zero indexing overhead, and DuckDB's cache-efficient hash aggregation.

6 Comparison with MySQL and MongoDB

6.1 Empirical Comparison with MySQL 9.6

MySQL 9.6 was benchmarked on the same hardware with the same OULAD data. The studentVle table was loaded into InnoDB using the default engine configuration. No query-level optimizations were applied to either database.

Several factors contribute to MySQL's slower analytical performance. First, row-oriented storage means each page read from disk contains all columns of each row, so a 2-column query reads 7 columns of data. Second, InnoDB's buffer pool is designed for OLTP workloads with frequent primary key lookups, not full table scans. Third, MySQL's iterator execution model processes one row per function call, incurring millions of extra function calls for a 10.6M-row scan.

For Q4 and Q5, the speedup narrows to 4.3x–20x because JOIN operations in both databases involve hash table construction overhead that partially masks the column pruning advantage. DuckDB's hash joins are also vectorized, processing 2048 tuples per build or probe operation, which maintains a significant advantage over MySQL's row-at-a-time execution.

6.2 Theoretical Comparison with MongoDB

MongoDB is a document-oriented NoSQL database that stores data as BSON documents. While MongoDB supports the aggregation pipeline with $group, $sort, and $lookup stages, its storage model differs fundamentally from DuckDB's. Documents bundle all fields together, so reading a subset of fields still requires deserializing the full document. MongoDB's aggregation pipeline processes documents one at a time, without vectorized batch processing.

For the OULAD workload, MongoDB would face two challenges. The 7-table relational schema would need to be denormalized into single documents or joined via $lookup, which is significantly slower than DuckDB's hash joins on large collections. The 10.6M fact table entries in studentVle would require either a single massive collection or manual sharding. MongoDB's strengths are schema flexibility and horizontal scalability, not high-throughput aggregation over normalized relational data.

7 Limitations, Lessons Learned, and Future Work

7.1 Limitations

- Single-node only: DuckDB cannot distribute queries across machines. Datasets exceeding available RAM or local disk require sharding or a distributed system. At institutional scale with billions of events, DuckDB would need data partitioning or complementary infrastructure.
- No concurrent access: DuckDB opens the database file exclusively. Multi-user environments require each analyst to maintain a separate copy or use a client-server system in parallel.
- No secondary indexes: DuckDB does not support B+ tree or hash indexes. Point lookups (WHERE id = X) fall back to full table scans with zonemap pruning. Workloads mixing point queries with aggregations need PostgreSQL or MySQL.
- Dataset scope: The OULAD dataset, while containing 10.6M rows, represents a single institutional context. Results may not generalize to larger or differently distributed datasets. Scaling to 1 billion rows would test DuckDB's memory and single-node limits.
- MySQL default configuration: MySQL was tested with default InnoDB settings. Tuning the buffer pool, using compressed row format, or disabling ACID guarantees during bulk load could reduce the performance gap. The benchmark reflects default MySQL performance, which is the experience most users encounter.

7.2 Lessons Learned

- Column pruning dominates performance: The 618x speedup on Q1 demonstrates that eliminating I/O is the most effective optimization for aggregation queries. DuckDB achieves this automatically without query hints, indexes, or schema denormalization.
- Vectorized execution adds real value: While column pruning explains most of Q1–Q3 speedups, vectorized hash joins in Q4–Q5 contribute meaningfully. DuckDB builds and

probes hash tables using vectorized loops, reducing per-tuple overhead compared to MySQL's tuple-at-a-time model.

- Tool selection depends on workload: DuckDB is not a general MySQL replacement. For transactional workloads, row inserts, or point queries, MySQL remains the better choice. DuckDB's value emerges specifically for analytical workloads where aggregation queries dominate.
- Real data reveals real patterns: The OULAD queries uncovered actionable educational insights. Students with withdrawn or failed outcomes show 3 to 5 times fewer VLE clicks than passing students. Low VLE engagement in the first two weeks correlates with higher dropout rates. These patterns, discovered through aggregation-heavy queries, demonstrate the practical value of fast analytical database processing.

7.3 Future Work

Several extensions could build on this project. A deeper comparison with ClickHouse, a distributed columnar OLAP system, would test whether DuckDB's architectural simplicity provides performance advantages or limitations at larger data volumes. Adding time-series analysis queries (window functions for moving averages, student trajectory clustering) would exercise DuckDB's window operator, which uses segment-tree accumulation optimized for vectorized execution. Finally, integrating DuckDB with a real-time LMS API feed would test whether the analytical speedup can be translated into near-real-time learning analytics dashboards that update as new clickstream data arrives.

8 Project Links

- GitHub repository: https://github.com/rosiexiu/dsci551-duckdb-project
- OULAD dataset (Kaggle): https://www.kaggle.com/datasets/anlgrbz/student-demographics-online-education-dataoulad
- DuckDB official site: https://duckdb.org

9 References

[1] H. Muhleisen and M. Raasveldt. 2019. DuckDB: an Embeddable Analytical Database. In Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19). Association for Computing Machinery, New York, NY, USA, 1981–1984. https://doi.org/10.1145/3299869.3320212

[2] P. Boncz, M. Zukowski, and N. Nes, "MonetDB/X100: Hyper-Pipelining Query Execution," in Proceedings of the 2nd Biennial Conference on Innovative Data Systems Research (CIDR), Asilomar, CA, 2005, pp. 225-237.

[3] J. Kuzilek, M. Hlosta, and Z. Zdrahal, "Open University Learning Analytics Dataset," Scientific Data, vol. 4, art. no. 170171, 2017, doi: 10.1038/sdata.2017.171.

[4] A. Lamb et al., “The Vertica Analytic Database: C-Store 7 Years Later,” Proceedings of the VLDB Endowment, vol. 5, no. 12, pp. 1790–1801, 2012.

[5] D. J. Abadi, P. A. Boncz, and S. Harizopoulos, “Column-Oriented Database Systems,” Proceedings of the VLDB Endowment, vol. 2, no. 2, pp. 1664–1665, 2009.

Appendix A: AI Usage Disclosure

Claude (Anthropic, Opus 4.6) was used during this project in the following capacities:

1. Code assistance: The AI helped debug syntax errors in the Jupyter notebook, refactor hardcoded file paths to use relative paths, and generate boilerplate code for benchmark measurements.

2. Report revising: The AI was used to suggest structural outline, expand technical explanations. The author wrote, reviewed, revised, and takes full responsibility for all content, analysis, and conclusions presented in this report.

3. Citation management: The AI helped format in-text citations in IEEE style. All references were selected verified by the author.

The AI tool was not used to fabricate data, generate benchmark results, or replace the author’s own understanding of the material. All queries, analyses, and interpretations reflect the author’s independent work. The AI served solely as a productivity tool, analogous to using spell-check, grammar checkers, or template-based formatting in traditional writing workflows.

In accordance with USC’s academic integrity policies, the use of AI assistance is disclosed transparently to maintain the integrity of the learning process and the submitted work.